Piano Man

School of Engineering and Applied Sciences
Harvard University

Cambridge, MA

Description: An arcade game inspired by the popular mobile app Piano Tiles, designed using circuitry and programmed in Arduino IDE.

By: Anaya Bindal, Diego Cuevas Vega, Sophia Hwang

Class: Engineering Science 50
Instructors: Vijay Janapa Reddi and Marko Loncar

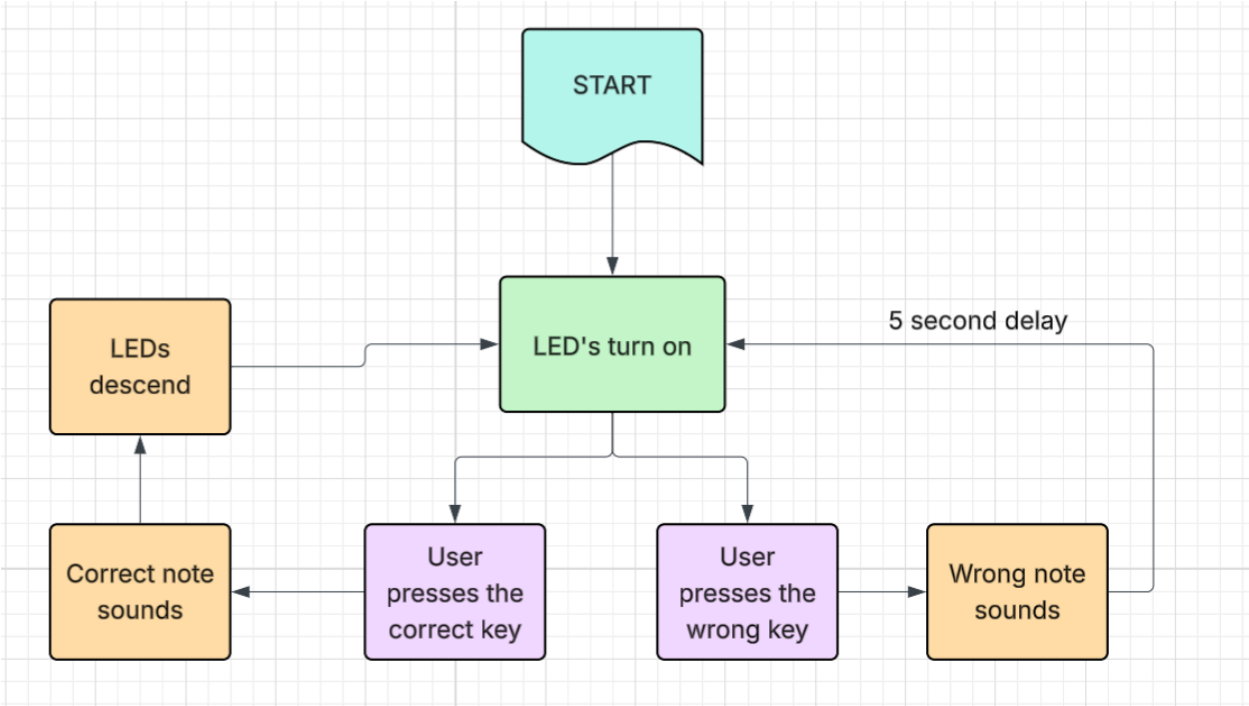
May 2026

Abstract:

Our project is a real time interactive arcade game inspired by the mobile app called Piano Tiles. The physical system is a 4x4 grid that consists of 16 LED pushbuttons, which simulate a “falling tiles” animation across the four columns. A player interacts only with the four buttons of the frontmost row, and the system operates on a loop where the player must press the button that illuminates. The three rows above the active row are used to notify the player of the upcoming tiles, mirroring the app experience. If the user presses the correct button, the tile grid advances (each row shifts down) and plays a note through a speaker. If the user presses the incorrect button, a penalty tone is triggered, pausing the game for 3 seconds. The gameplay duration is 45 seconds, and a player’s score is transmitted wirelessly to a web-based leaderboard through an ESP32 which is updated in real time. The system uses an Arduino MKR Zero for the game logic and button pressing control, an ESP32 for WiFi communication and scoring, an op amp for audio, and a buck converter for converting the DC power supply. The project incorporates input processing, analog signal amplification, LED row-column scanning, UART communication, and human system interaction.

System Overview:

The system incorporates key concepts from ES50:



Physical button press→ Arduino digital read → game logic decision → LED grid update + audio tone + serial score recording → ESP32 broadcast → browser update

1. User input: Performed through the 4 arcade style pushbuttons that map to Arduino digital input pins, detecting player presses.
2. Game Logic: The Arduino maintains an array that detects incorrect and correct button presses.
3. Visual output: A 4x4 LED matrix is driven by transistors in a row-column configuration and it is powered to light up with a 12V power supply.
4. Audio output: A TL081 op-amp circuit amplifies the Arduino's speaker pin output and drives the speaker.
5. Score display: The ESP32 creates its own WIFi network for the leaderboard web page which receives scores and incorrect button presses from the Arduino using a UART connection in real time.

Physical Components:

1. Microcontrollers:

The game uses an Arduino MKR Zero microcontroller which handles real time logic decisions. It reads the button inputs from pins 4, 5, 3, and 2 on the Arduino and controls the 4x4 LED matrix with the LED rows { 6, 9, 10, 11} and the LED columns { 8, 7, 12, 0} respectively, which generates audio tones on its speaker pin 1. It also sends incorrect button press events via the ESP32 Serial1 port. It operates at 5V and interfaces with the 12 V LED hardware using NPN transistors and PMOS and NMOS mosfets.

In order to produce a leaderboard, we used the ESP32 microcontroller to create a local network and host a webpage. This way we avoided connecting any of our computers directly to the circuit. The ESP32 chip created a WiFi network named "PianoTiles_Net", which we then connected our computers to. With this and the IP address the ESP32 provided, we could see the leaderboard on an external monitor. In order to achieve the real-time update, the MKR Zero and the ESP32 communicated through the UART protocol, which enabled the MKR Zero to notify the ESP32 of correct/incorrect button presses.

2. LED Matrix:

The game uses 16 LED pushbuttons that are set up in a 4x4 grid. The rows were scanned one at a time, where the updateDisplay( ) function in the code turned all LEDs off and activated a single row at a time. The pushbuttons of the front row are bigger than all the other 12. Additionally, by creating a small delay in the activation of the LEDs per row, we were able to create the illusion

of simultaneous lighting while preventing any overheating. The LED rows were powered with 12 V through transistors and PMOS mosfets while the columns were connected to ground and the NMOS mosfets. The row and column control signals were controlled through the MKR Zero's pins. A base resistor limited the current sent and a resistor in series limited the high current through the LEDs at the high voltage.

The queue of lights was done through an array, where grid[0] was the pressable row and grid[1], grid[2], and grid[3] were the previous rows from bottom to top. A correct press allowed the queue to shift down and a new random LED was assigned to be ON on row grid[3].

In order to reduce confusion and allow for a swifter debugging process, we decided to add a separate breadboard to the side which contained all the connections for the rows and columns. This was done so we could quickly localize any connection mishaps by tracing the mistake directly to the breadboard. Additionally, this helped clear the main breadboard and provided visual clarity.

3. Pushbutton Input:

The four arcade buttons in the first row were wired to the Arduino pins. The buttons were active-HIGH and returned HIGH when pressed. The game checked whether the pressed button index matched the active tile. If it matched, the correct note was played and the queue advanced. If a wrong button was pressed, the wrongNote( ) function activated.

4. Audio Circuit:

The audio was connected as follows: the MKR Zero speaker pin outputted a square wave via the Arduino tone( ) function. Then the TL081 was wired as a non-inverting amplifier to boost the signal to a level sufficient to drive the speaker. The op-amp was powered from the 12V supply rail, the non-inverting input (+) received the Arduino signal, and the feedback network helped to set the voltage gain. The speaker converted the amplified electrical signal to sound.

The musical notes were defined as frequencies in an integer array with the notes from a given song converted to frequencies. The notes all had a duration of 200 ms. The playNote() function called tone(SPEAKER_PIN, notes[currentNote], durations[currentNote]), advanced currentNote (wrapping around modularly), then called noTone() after 100 ms. The wrong-press penalty sound was a 100 Hz tone lasting 2 seconds.

5. Power Supply:

The system was powered from an external DC supply and employed a buck converter that stepped the voltage down to provide sufficient power to microcontrollers. The LED matrix and op amp operated directly from the 12V. Ground references were all connected.

Software:

The finalized code was programmed in Arduino IDE and consisted of multiple self-defined global variables and functions.

1. The beginning of the code initializes values for LED_ON, LED_OFF, and SPEAKER_PIN, defines the corresponding led row, column, and button pins on the Arduino, defines an array of note frequencies (called *notes*), initializes the game score, and initializes the timer.
2. The setAllOff function sets all of the LEDs to turn off.
3. The playNote function plays the current note depending on the index value (defined by the global variable *currentNote*) from the array *notes*.
4. The wrongNote function plays a wrong note sound and sets all LEDs off for 3 seconds.
5. The updateDisplay function manages the visual representation of the board by utilizing multiplexing to rapidly cycle through each row. It turns each row on at a time and cycles through all the rows with a 2 ms delay between each transition, switching between each row so quickly that the LEDs appear to be on simultaneously rather than blinking.
6. The setup function sets up the board, identifying the first four random buttons to be turned on in each row, starts the time counter, sets all the LEDs off, and begins communication with the ESP32.
7. The loop function runs the game by increasing the score, playing the note, and sending the updated score to the ESP32 when the correct note is played, controlling the wrong note logic when the wrong LED is pressed, and turning the game off when the timer ends.

ESP32 code:

1. The first part of the code imports the necessary libraries to work with WiFi in the ESP32
2. The ESP32 creates the local WiFi named “PianoTiles_Net”, with the password “password123”.
3. The ESP32 is given the HTML code to host the leaderboard website. Here lies all the code for the website and to update according to what the MKR Zero sends.
    a. The most important section is the function initWebSocket() which allows the HTML to update as well as the websocket.onmessage arrow function which identifies the score and miss data from the single data string.
4. In the setup, it sets the Serial2 channel ready for UART communication with a 9600 baud rate. It uses the pins 3 and 1 for RX and TX. SERIAL_8N1 describes the data format- 8 data bits, and 1 stop bit.
5. It initializes the WiFi, and runs the html.
6. In the loop void, it continuously updates the information and incorporates it into the web socket–the web socket shares it with all the connected devices. It takes it all as a single data string.

Circuits:

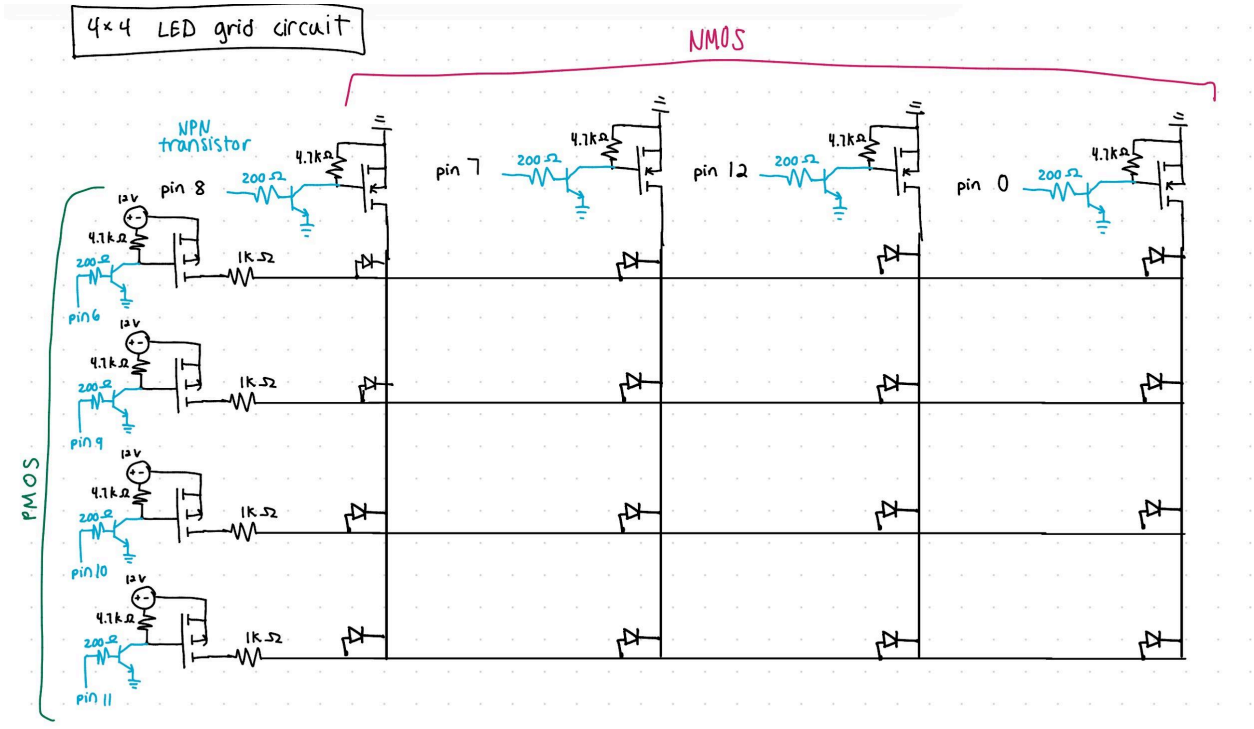


Figure 1: Circuit schematic of 4x4 LED grid with NPN transistors, and NMOS and PMOS mosfets.

Figure 2: Sound circuit schematic

The circuit employs an LM386 op amp in order to amplify the frequencies of the speaker with the following pin out. Pin 2 is connected to ground, Pin 3 is connected to a pin of the Arduino, Pin 4 is connected to ground, Pin 6 is connected to a buck converter (which also is to ground), and Pin 5 is connected to the positive end of the speaker. All the grounds are common throughout all circuit components.

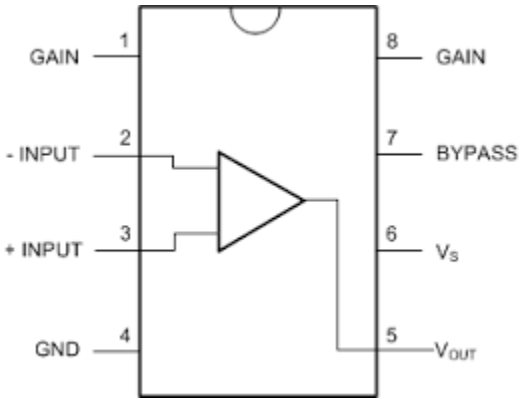


Figure 3: LM386 Pinout

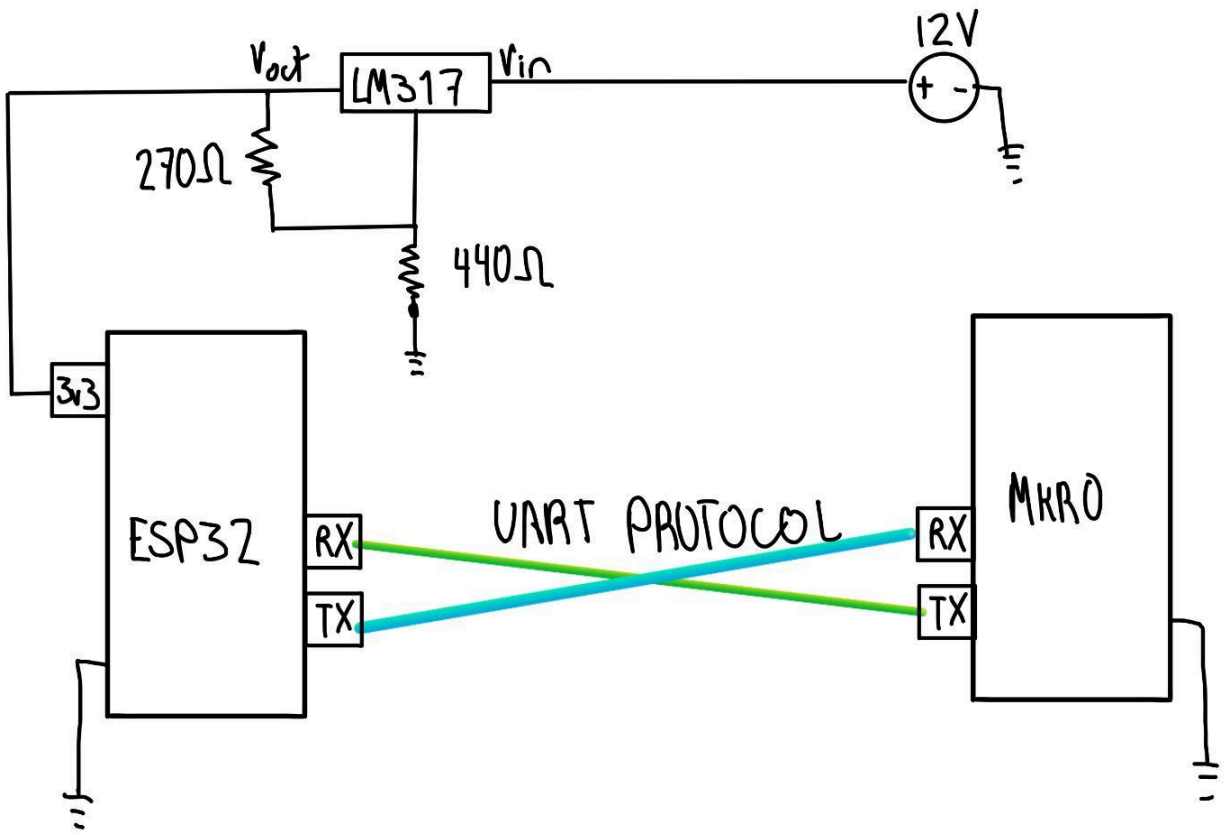


Figure 4: ESP32 circuit showcasing the UART communication wiring between the ESP32 and the MKR0. Additionally, it shows the buck converter circuit used to go from 12V to 3.3V.

Alternative Designs:

Arduino vs. ESP32: We originally planned to use a ESP32 to drive out game logic, but we switched to an Arduino and used the ESP32 for wireless communication instead. We decided this because combining all of these functions on a single microcontroller seemed to be unreliable.

Penalty design: We originally planned to employ a 5 second penalty, but changed it to 3 seconds after trial and error since testing found that 5 seconds felt too long at the game speed we chose.

ESP32 vs. LCD Display: We considered using an LCD display for the leadership display rather than the ESP32 WiFi network and display on our computer. If we had more time and budget we may have implemented this, but given the time crunch and $100 budget we chose to use an HTML display.

Limitations:

Core limitations of this project was our $100 project and 2.5 week time crunch. Another limitation was that the available voltage regulator heated up too quickly given the large current requirement of the ESP32. This made us rely on always having replacements for it, and having to constantly turn off the device because otherwise the voltage regulator would overheat. Additionally, the leaderboard was stored in the browser, so the score only would save per device and if the browser was completely cleared/reset, the leaderboard wouldn't save. Additionally, the audio quality of our op-amp+speaker wasn't the best and if we tried to use higher notes, there was noticeable distortion. Hence, we had a limited range of frequencies and notes to use.

Experimental Results:

We conducted a large amount of testing at each stage of our circuit as suggested by course staff. By ensuring that at each step individual components like the LED lights, rows, columns, or sound elements worked, we were able to debug more effectively at later stages of the project. We created test code for the LED matrix to ensure that the lights were able to light up and to make sure LED ghosting was not occurring. We also wanted to make sure that the correct tones and frequencies were being produced by the MKR Zero, which we tested by using simple harmonic scales against a piano before using more complicated songs. The LM386 op-amp circuit also produced a clean amplified output observed by an oscilloscope that showed no clipping. We made sure that the web leaderboard showed live updates as the buttons were pressed through simulation code ensuring no dropped messages.

Project Photos:

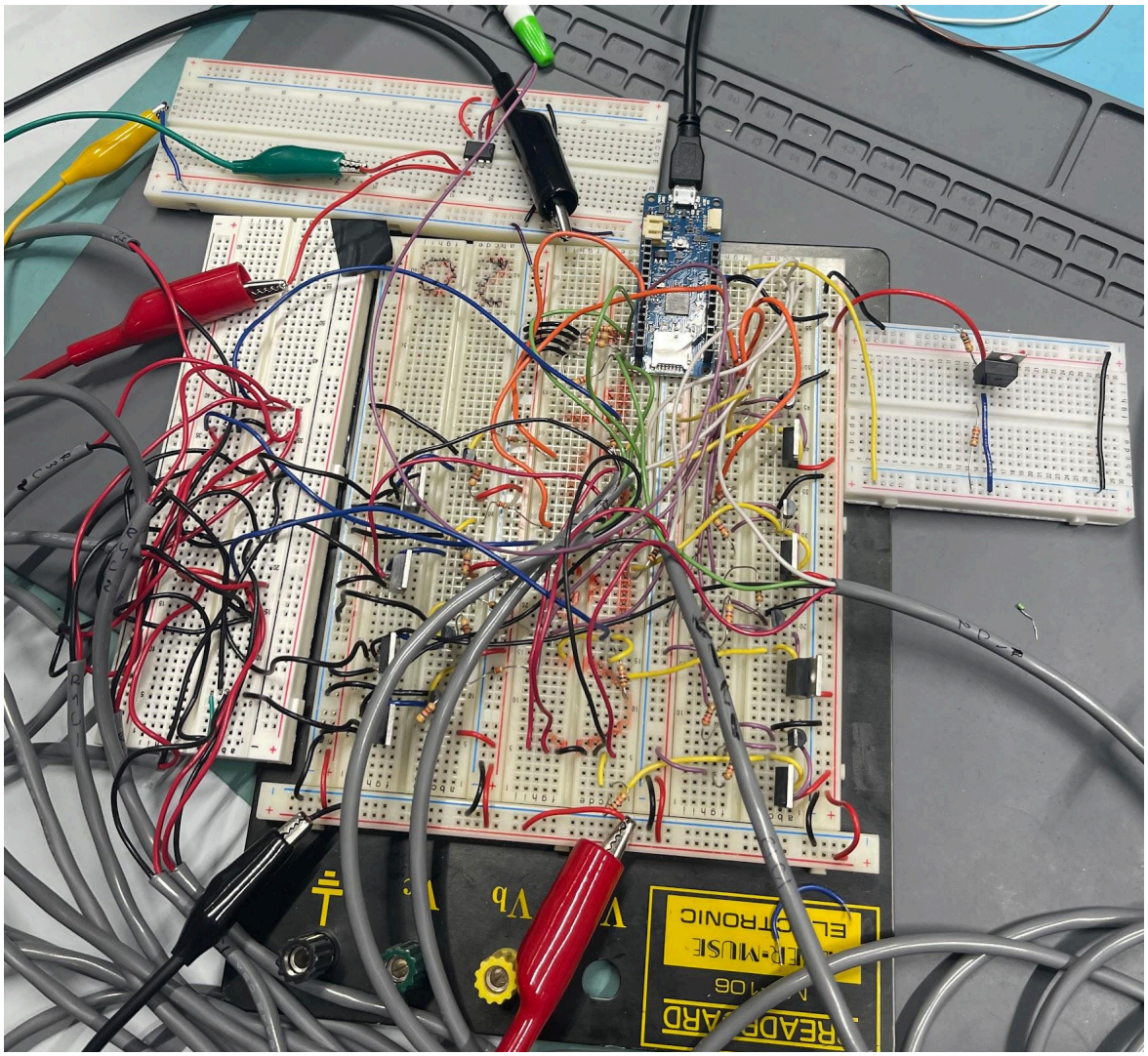


Figure 4: Breadboard circuit of 4x4 grid circuit with audio amplifier and buck converter.

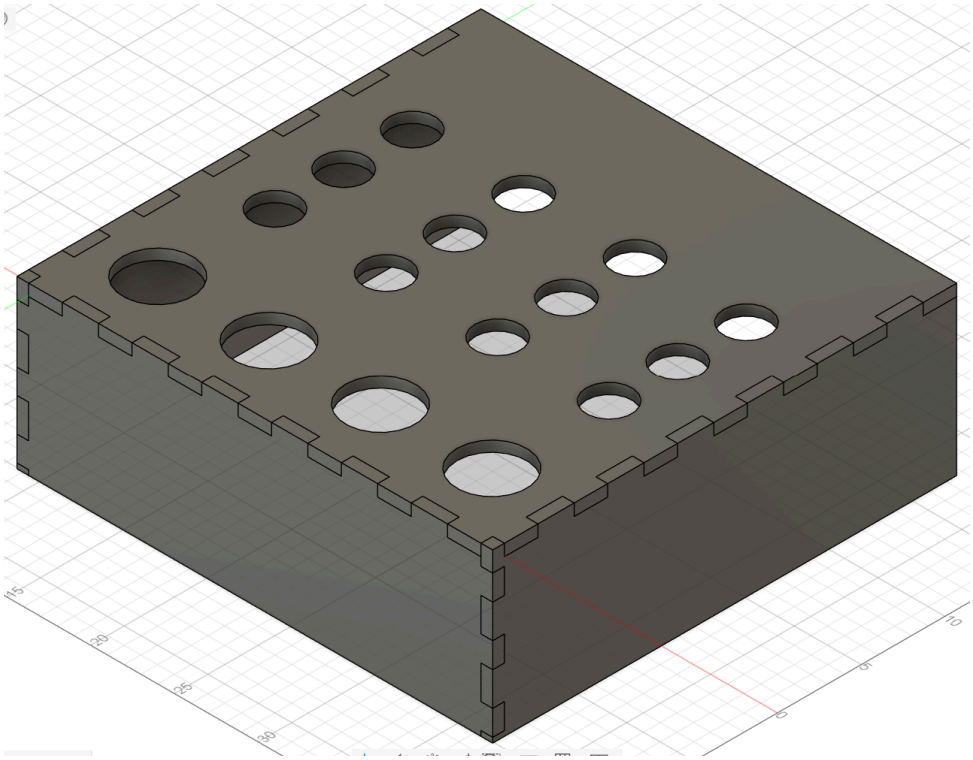


Figure 5: This was the 3D model of the acrylic enclosure used as a base for the whole project. It was first laser cut in cardboard and then using the acrylic we had. This was done in Fusion.

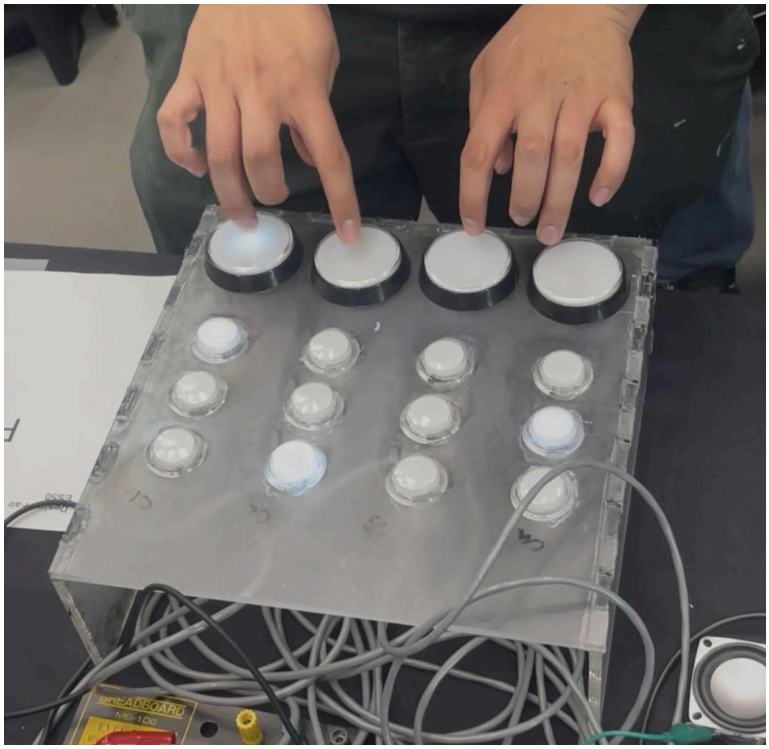


Figure 6: 4x4 box and LED array with the game activated (being played by Diego).

Bill of Materials:

Item Name	Vendor	Vendor Part Link	Part#	Quantity	Total price
uxcell Game Push Button 33.5mm Round 12V LED Illuminated Push Button Switch with Micro Switch for Arcade Video Games White 6pcs	Amazon		a19062100ux0358	2	30.78
uxcell Game Push Button 60mm Round 12V LED Illuminated Push Button Switch with Micro Switch for Arcade Video Games White 5pcs	Amazon	LED_Button	a19062100ux0206	1	18.89
ESP 32	Amazon	https://a.co/d/08jMYfiW		1	16
Gikfun 2" 4Ohm 3W Full Range Audio Speaker Stereo Woofer Loudspeaker for Arduino (Pack of 2pcs) EK1725	Amazon	Speaker_Amazon	LYSB01CHYIU26-ELECTRNCS	1	11.49
5 Pack LM2596 DC to DC Buck Converter 3.0-40V to 1.5-35V Adjustable Voltage Regulator Electronic Voltage Stabilizer Power Supply Step Down Module	Amazon	BoostConverter	SY-LM2596	1	7.99
1K Ohm resistors	EE Lab			4	
4.7K Ohm resistors	EE Lab			8	
200 Ohm resistors	EE Lab			8	
wires	EE Lab				

breadboards	EE Lab				
IRFZ24 NMOS	EE Lab			4	
IRF9Z24 PMOS	EE Lab			4	
2N 3904 transistor	EE Lab			8	
ULN2003 ULN2003AN ULN2003A ULN2003APG High-Voltage High-Current Seven Darlington Transistor Arrays NPN IC DIP-16 (Pack of 10pcs)	Amazon	https://a.co/d/05KpuspU	ULN2003A	1	6.99
Sheet of Acrylic	Amazon	https://a.co/d/0f6hphg5	12 x 12 x 1/4”	1	
Power Supply	EE Lab			1	
Banana to Alligator cords	EE Lab			4	
LM386 Op Amp	EE Lab			1	
Alligator to Alligator cords	EE Lab			2	

Acknowledgements:

We’d like to thank Marko and VJ for providing us with the necessary information, drive, and enthusiasm in creating this project. We’d also like to thank our lab TFs (especially Champa, Donald, and Renee) for all their support in debugging, building, and even exploding laptops. We also acknowledge Claude AI for helping debug our code. Thank you to everyone who helped us bring our engineering dreams to life.

Contributions:

Diego designed the network between the ESP32 and leaderboard simulation, as well as worked on creating and implementing the buck converters for correct power sourcing throughout the circuit. Diego also worked on debugging the code and the LED matrix. He also cut the LED matrix box from acrylic and designed the box shown in Figure 6.

Anaya developed the audio portion of the circuit and worked on creating the code for all the songs and audio circuit including delays and connecting it with Sophia’s LED matrix to detect

button presses. She also created the HTML leaderboard code, soldered circuit components. And assisted in debugging.

Sophia built the circuit controlling the 4x4 LED matrix shown in Figure 1 and 4 and developed the falling tile, wrong note logic, and LED sequencing game code. She contributed largely in debugging the LED circuit and integrating the other components into the overall structure of the project.

APPENDIX A: Master Code:

```
#define LED_ON 0
#define LED_OFF 1
#define SPEAKER_PIN 1

// the row, column, and button pins of the Arduino IDE
const int ledRows[] = {6, 9, 10, 11};
const int ledCols[] = {8, 7, 12, 0};
const int buttonPins[] = {4, 5, 3, 2};
int grid[4];

// MUSIC NOTES FOR THE SONG
int notes[] = {294, 392, 466, 392, 370, 392, 440, 392, 311, 349, 392, 294,
262, 233, 233, 220, 220, 294, 196, 294, 392, 466, 587, 466, 440, 466, 523,
466, 622, 699, 784, 622, 587, 622, 699, 587, 523, 587, 622, 523, 466, 523,
587, 466, 523, 466, 466, 440, 440, 587, 392};
int numNotes = sizeof(notes) / sizeof(notes[0]);
int num = 51;
int durations[51];
int currentNote = 0;


// SCORING FOR LEADERBOARD
int gameScore = 0;

// TIMER
unsigned long startTime;
const unsigned long gameLimit = 45000;
bool gameActive = true;
```

```
// sets all LEDs off
void setAllOff() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(ledRows[i], !LED_OFF);
    digitalWrite(ledCols[i], LED_OFF);
  }
}

// plays the current note
void playNote() {
  tone(SPEAKER_PIN, notes[currentNote], durations[currentNote]);
  delay(100);
  noTone(SPEAKER_PIN);
  currentNote = (currentNote + 1) % numNotes;
}

// plays the wrong note sequence
void wrongNote() {
  Serial1.println("MISS"); // Send "MISS" to ESP32
  tone(SPEAKER_PIN, 100, 3000);
   // starts the time penalty of 3 seconds when the wrongNote function is
called
  unsigned long penaltyStart = millis();
  while(millis() - penaltyStart < 3000) {
    setAllOff();
  }
  noTone(SPEAKER_PIN);
}

// updates the display of the game
void updateDisplay() {
  // iterate through all 4 rows
  for (int i = 0; i < 4; i++) {
    // clear all LEDs
    setAllOff();
    //activate the current row and column
    digitalWrite(ledRows[i], !LED_ON);
    digitalWrite(ledCols[grid[i]], LED_ON);
    delay(2);
```

```
  }
}


void setup() {
  Serial.begin(9600);
  Serial1.begin(9600); // Start communication with ESP32

  // random generator for notes
  randomSeed(analogRead(0));

  // initialize the starting game board with note durations to 200 ms
  for (int i = 0; i<num; i++) { durations[i] = 200; }
  for (int i = 0; i < 4; i++) {
    // set the button pins as input and the rows and columns as outputs
    pinMode(buttonPins[i], INPUT);
    pinMode(ledRows[i], OUTPUT);
    pinMode(ledCols[i], OUTPUT);
    grid[i] = random(0, 4);
  }
  // set the speaker pin as output
  pinMode(SPEAKER_PIN, OUTPUT);
  setAllOff();
  // start the time
  startTime = millis();
}


void loop() {
  // check if the game is still active (if  45 seconds has elapsed)
  if (gameActive) {
    if (millis() - startTime >= gameLimit) {
      gameActive = false;
      setAllOff();
      Serial.println("GAME OVER - Time is up!");
      return;
    }

    // continuously refresh the LED grid
    updateDisplay();
```

```
    // determine correct button to press
    int targetButtonIndex = grid[0];
    // if the player presses the correct button
    if (digitalRead(buttonPins[targetButtonIndex]) == HIGH) {
      gameScore++; // Increase score
      Serial1.print("SCORE:"); // Send updated score to ESP32
      Serial1.println(gameScore);

      playNote();
      // shifts the tiles down by each row
      for (int i = 0; i < 3; i++) { grid[i] = grid[i + 1]; }
      grid[3] = random(0, 4);
      // wait for the button to be released
           while (digitalRead(buttonPins[targetButtonIndex]) == HIGH) {
updateDisplay(); }
      delay(50);
    }
    // if player presses any of the wrong buttons
      else if ((digitalRead(buttonPins[(targetButtonIndex + 1) % 4]) ==
HIGH) || (digitalRead(buttonPins[(targetButtonIndex + 2) % 4]) == HIGH) ||
(digitalRead(buttonPins[(targetButtonIndex + 3) % 4]) == HIGH)){
      wrongNote();
    }
  } else {
    // if the game is over, turn all LEDs off
    setAllOff();
  }
}
```

APPENDIX B: Leaderboard Code:

```
#include <WiFi.h>
#include <AsyncTCP.h>
#include <ESPAsyncWebServer.h>
```

```
const char* ssid = "PianoTiles_Net";
const char* password = "password123";

AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
    <title>Piano Tiles Leaderboard</title>
    <style>
         body {background: #000000; color: white; font-family: sans-serif;
display: flex; flex-direction: column; align-items: center; padding: 40px
20px; }
        h1 { font-size: 36px; margin-bottom: 8px; }
        .metrics { display: flex; gap: 24px; margin: 32px 0; }
         .metric { background: #1a1a1a; border-radius: 12px; padding: 24px
32px; text-align: center; min-width: 120px; }
           .metric-label { font-size: 12px; color: #888; text-transform:
uppercase; letter-spacing: 0.1em; margin-bottom: 8px; }
        .metric-value { font-size: 48px; font-weight: 500; }
        .score-value { color: #5dcaa5; }
        .hits-value { color: #97c459; }
        .misses-value { color: #f09595; }
        .feed { width: 100%; max-width: 500px; margin-bottom: 32px; }
            .feed-title { font-size: 12px; color: #888; text-transform:
uppercase; letter-spacing: 0.1em; margin-bottom: 12px; }
            .feed-item { display: flex; justify-content: space-between;
align-items: center; padding: 10px 16px; border-radius: 8px;
margin-bottom: 6px; font-size: 14px; }
        .feed-item.hit { background: #1a2e1a; color: #97c459; }
        .feed-item.miss { background: #2e1a1a; color: #f09595; }
        .leaderboard { width: 100%; max-width: 500px; margin-bottom: 32px;
}
             .lb-title { font-size: 12px; color: #888; text-transform:
uppercase; letter-spacing: 0.1em; margin-bottom: 12px; }
```

```
        .lb-row { display: flex; justify-content: space-between; padding: 10px 16px; background: #1a1a1a; border-radius: 8px; margin-bottom: 6px; font-size: 14px; }
       .lb-rank { color: #888; width: 24px; }
       .lb-name { flex: 1; padding-left: 12px; }
       .lb-score { color: #5dcaa5; font-weight: 500; }
        .name-row { display: flex; gap: 8px; margin-bottom: 24px; width: 100%; max-width: 500px; }
        input { flex: 1; padding: 10px 14px; border-radius: 8px; border: 1px solid #333; background: #1a1a1a; color: white; font-size: 14px; }
       .status { font-size: 13px; color: #888; margin-bottom: 24px; }
         .status .dot { display: inline-block; width: 8px; height: 8px; border-radius: 50%; background: #444; margin-right: 6px; }
       .status .dot.connected { background: #97c459; }
    </style>
</head>
<body>
    <h1>Piano Tiles</h1>
          <div  class="status"><span  class="dot"  id="dot"></span><span id="status-text">Disconnected</span></div>
    <div class="name-row">
          <input type="text" id="player-name" placeholder="Enter player name..." />
       <button id="save-btn" style="padding:10px 24px; border-radius:8px; background:#1a1a1a;     color:white;     border:1px     solid     #333; cursor:pointer;">Save score</button>
    </div>
    <div class="metrics">
           <div class="metric"><div class="metric-label">Score</div><div class="metric-value score-value" id="score-val">0</div></div>
            <div class="metric"><div class="metric-label">Hits</div><div class="metric-value hits-value" id="hits-val">0</div></div>
          <div class="metric"><div class="metric-label">Misses</div><div class="metric-value misses-value" id="misses-val">0</div></div>
    </div>
        <div  class="feed"><div  class="feed-title">Live  feed</div><div id="feed-list"></div></div>
     <div class="leaderboard"><div class="lb-title">Leaderboard</div><div id="lb-list"></div></div>
```

```
    <script>
        let score = 0, hits = 0, misses = 0;
        let gateway = `ws://${window.location.hostname}/ws`;
        let websocket;
                                                let      leaderboard      =
JSON.parse(localStorage.getItem('pt_leaderboard') || '[]');

        function initWebSocket() {
            websocket = new WebSocket(gateway);
            websocket.onopen = () => {
                        document.getElementById('dot').className  =  'dot
connected';
                document.getElementById('status-text').textContent = 'Live
Wireless Feed';
            };
            websocket.onmessage = (event) => {
                let line = event.data.trim();
                if (line.startsWith('SCORE:')) {
                    score = parseInt(line.replace('SCORE:', ''));
                    hits = score;
                    updateUI('hit', 'Correct button!');
                } else if (line === 'MISS') {
                    misses++;
                    updateUI('miss', 'Wrong button');
                }
            };
        }

        function updateUI(type, msg) {
            document.getElementById('score-val').textContent = score;
            document.getElementById('hits-val').textContent = hits;
            document.getElementById('misses-val').textContent = misses;
            const div = document.createElement('div');
            div.className = 'feed-item ' + type;
                        div.innerHTML  =  `<span>${msg}</span><span>Score:
${score}</span>`;
            document.getElementById('feed-list').prepend(div);
             if (document.getElementById('feed-list').children.length > 8)
document.getElementById('feed-list').lastChild.remove();
        }
```

```
        function renderLeaderboard() {
                const sorted = [...leaderboard].sort((a, b) => b.score -
a.score).slice(0, 8);
             document.getElementById('lb-list').innerHTML = sorted.map((e,
i) => `
                                                <div   class="lb-row"><span
class="lb-rank">${i+1}</span><span   class="lb-name">${e.name}</span><span
class="lb-score">${e.score}</span></div>
            `).join('');
        }

        document.getElementById('save-btn').onclick = () => {
                                                    const    name    =
document.getElementById('player-name').value.trim();
            if(name) {
                leaderboard.push({name, score});
                                    localStorage.setItem('pt_leaderboard',
JSON.stringify(leaderboard));
                renderLeaderboard();
                document.getElementById('player-name').value = '';
            }
        };

        window.onload = () => { initWebSocket(); renderLeaderboard(); };
    </script>
</body>
</html>
)rawliteral";

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, 3, 1);

  WiFi.softAP(ssid, password);
  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });
  server.addHandler(&ws);
  server.begin();
```

```
}

void loop() {
  if (Serial2.available()) {
    String data = Serial2.readStringUntil('\n');
    ws.textAll(data);
  }
  ws.cleanupClients();
}
```

DISCLAIMER

We give full permission to use the code and schematics/diagrams that we created, and share any photos and videos of our product.